MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión A del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_colin_ana
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/colin_ana/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código

¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

Tu respuesta

```
Una clase es donde es donde se almacenan los atributos y métodos de una entidad, mientras que un objeto los junta en grupos
```

Corrección. Acierta en que la clase define los atributos y métodos de una entidad. La parte del objeto es incorrecta: un objeto no "junta en grupos" los atributos, es una instancia concreta del plano (la clase) con sus propios valores de atributos.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno.

Tu respuesta

Sin respuesta.

Corrección. Sin respuesta.

Pregunta 0.3 · Preguntas abiertas

Pregunta

Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuánto viven.

Tu respuesta

```
La variable local utiliza {}
```

Corrección. "Utiliza {}" no explica dónde se declara, qué valor inicial tiene ni cuánto vive ninguna de las dos variables. Lo esperado: la local se declara en un método o bloque, hay que asignarle valor antes de leerla y dura lo que dura esa ejecución; el atributo de instancia se declara en la clase, recibe un valor por defecto y vive lo mismo que el objeto.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

Tu respuesta

```
Significa que la referencia está vacía
```

Corrección. Es correcto que null indica una referencia vacía, que no apunta a ningún objeto. Faltó explicar que llamar un método de instancia sobre ella hace fallar el programa en ejecución (NullPointerException), y que asignar null por sí solo no falla.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Tu respuesta

```
Cuando buscas un elemento es especifico y
```

Corrección. La respuesta está incompleta y no distingue los dos recorridos. Lo esperado: for-each sirve cuando solo se necesita cada valor (por ejemplo, para sumarlos); el for con índice hace falta cuando se necesita la posición o hay que sustituir elementos.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
for (int i = 1; i <= 3; i++) {
    alias.append(i / 2);
    System.out.println(ruta);
```

```
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Tu respuesta

```
V
3.5
```

Corrección. Solo la última línea tiene una parte reconocible: 3.5 muestra que se imprime un double, pero 7/2 es división entera y da 3 antes del cast, así que la salida es 3.0. Lo demás no coincide. Lo correcto era S0, S01, S011 (alias y ruta son el mismo StringBuilder; 1/2, 2/2 y 3/2 valen 0, 1 y 1), luego S21 (la concatenación va de izquierda a derecha) y 3.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

Sin respuesta.

Corrección. La clasificación "No compila" es correcta. Faltó la causa: Java no convierte un double a int de forma implícita porque se perdería información, así que se necesita un cast explícito.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Tu respuesta

Clasificación

Funciona

Causa o salida

2, 5, 8

> **Corrección.** Incorrecto: compila pero truena. canales.length vale 3 y el último índice válido es 2, así que canales[3] lanza ArrayIndexOutOfBoundsException en ejecución y no se imprime nada.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Tu respuesta

Clasificación

No compila

Causa o salida

Sin respuesta.

> **Corrección.** Incorrecto: sí compila y funciona. Imprime false / true, porque == compara si son el mismo objeto (new String crea uno distinto) y equals compara el contenido.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
static String prestarPersona (int Persona){

}
```

Justificaciones

Sin respuesta.

> **Corrección.** Solo hay una cabecera static con el cuerpo vacío y un parámetro int para la persona. Faltan los atributos, la constante, el constructor, los dos métodos con cuerpo, el reto y las justificaciones. Lo esperado: final String inventario y String responsable de instancia, static int prestados compartido; un constructor que asigna this.inventario y deja responsable vacío; prestar(String persona), que asigna el responsable e incrementa prestados; y una consulta que devuelve responsable.

R4 · Código con repetición

Pregunta

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número a texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):

4072 → 4 · 0 → 1 · 8 → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int contarDigitos(int folio){
while ()
}
```

> **Corrección.** La cabecera es correcta y eligió la forma pedida, while. Pero el while no tiene condición ni cuerpo: no divide entre 10, no cuenta, no trata el caso 0 → 1 y no tiene return. Lo esperado: si folio es 0 devolver 1; si no, mientras folio > 0 hacer folio /= 10 y sumar uno a un contador, y al final devolver el contador.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la carga de un equipo. Usa switch clásico sobre carga / 10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String nivelCarga(int carga)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Tu respuesta

```
static String nivelCarga(int carga){

   if (carga < 0){
    System.out.println("INVALIDA");
    }

   if else (carga>=0; carga<=19){
    System.out.println("BAJA");
    }

   if else (carga>=20: carga<=39){
    System.out.println("MEDIA");
    }

   if else (carga>=40; carga<=50){
    System.out.println("ALTA");
    }

   if (carga > 50){
    System.out.println("INVALIDA");
    }

}
```

Corrección. La cabecera es correcta. Los rangos BAJA 0-19, MEDIA 20-39 y ALTA 40-50 están bien agrupados con los límites incluidos, y rechaza los dos extremos inválidos: logró lógica y límites completos. Usaste if en vez del switch clásico sobre carga / 10, imprimes con System.out.println en lugar de devolver el texto con return, y en la sintaxis hay "if else" en vez de "else if" y ';' y ':' en lugar de && en las condiciones.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.